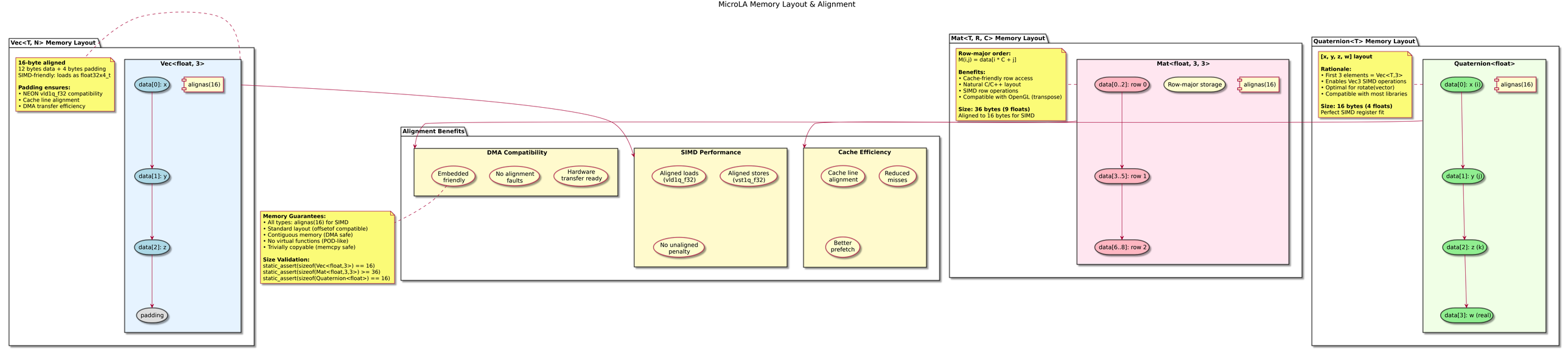
MicroLA Memory Layout & Alignment
Vec<T, N> Memory Layout
16-byte aligned
12 bytes data + 4 bytes padding
SIMD-friendly: loads as float32x4_t
Padding ensures:
• NEON vld1q_f32 compatibility
• Cache line alignment
• DMA transfer efficiency
Vec<float, 3>
data[0]: x
alignas(16)
data[1]: y
data[2]: z
padding
Memory Guarantees:
• All types: alignas(16) for SIMD
• Standard layout (offsetof compatible)
• Contiguous memory (DMA safe)
• No virtual functions (POD-like)
• Trivially copyable (memcpy safe)
Size Validation:
static_assert(sizeof(Vec<float,3>) == 16)
static_assert(sizeof(Mat<float,3,3>) >= 36)
static_assert(sizeof(Quaternion<float>) == 16)
Alignment Benefits
DMA Compatibility
Embedded friendly
No alignment faults
Hardware transfer ready
SIMD Performance
Aligned loads (vld1q_f32)
Aligned stores (vst1q_f32)
No unaligned penalty
Cache Efficiency
Cache line alignment
Reduced misses
Better prefetch
Mat<T, R, C> Memory Layout
Row-major order:
M(i,j) = data[i * C + j]
Benefits:
• Cache-friendly row access
• Natural C/C++ layout
• SIMD row operations
• Compatible with OpenGL (transpose)
Size: 36 bytes (9 floats)
Aligned to 16 bytes for SIMD
Mat<float, 3, 3>
data[0..2]: row 0
Row-major storage
alignas(16)
data[3..5]: row 1
data[6..8]: row 2
Quaternion<T> Memory Layout
[x, y, z, w] layout
Rationale:
• First 3 elements = Vec<T,3>
• Enables Vec3 SIMD operations
• Optimal for rotate(vector)
• Compatible with most libraries
Size: 16 bytes (4 floats)
Perfect SIMD register fit
Quaternion<float>
data[0]: x (i)
alignas(16)
data[1]: y (j)
data[2]: z (k)
data[3]: w (real)